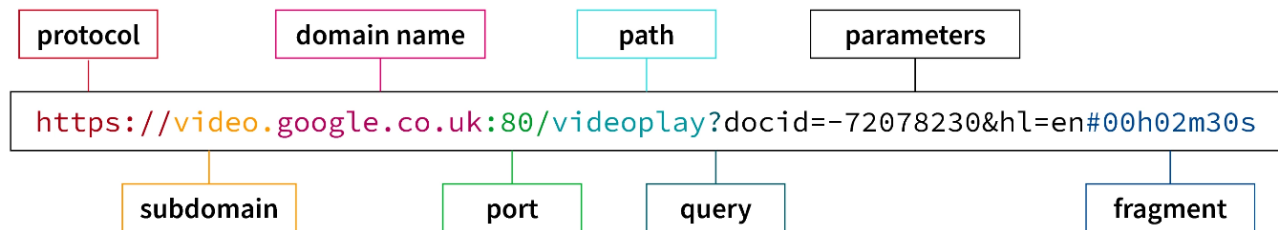


URL module

- ✓ The URL module in Node.js is used to parse and handle URLs efficiently.
- ✓ It allows developers to extract different parts of a URL such as protocol, hostname, port, pathname, and query parameters.
- ✓ This helps in web development for routing, handling requests, and processing user input.



The syntax for including the url module in your application:

```
var url=require("url");
```

The `url.parse()` method takes the url string as a parameter and parses it. The url module returns an object with each part of the url as property of the object.

Syntax

```
url.parse(urlString, parseQueryString)
```

- **urlString** → the full URL
- **parseQueryString** → boolean
 - false (default) → returns **string**
 - true → returns **object**

Behavior of query

The query property of `url.parse()` represents the **query string (key-value pairs) after ? in a URL**.

1. When `parseQueryString = false` (default)

- query is returned as a **raw string**
- No parsing is done

```
const url = require("url");  
const q = url.parse("http://localhost:3000/?a=10&b=20");  
console.log(q.query);
```

Output:

"a=10&b=20"

2. When `parseQueryString = true`

- query is parsed into an **object**
- Uses internal `querystring` parsing

```
const url = require("url");
const q = url.parse("http://localhost:3000/?a=10&b=20", true);
console.log(q.query);
```

Output:

```
{ a: '10', b: '20' }
```

Example-1

Write a Node.js program to **parse a URL** using the `url` module and display all its components **without parsing the query string**.

```
var u=require("url");
var addr="http://localhost:8080/default.html?year=2025&month=feb#test";
var q=u.parse(addr);
console.log(q);
```

Output:

```
Url {
  protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'localhost:8080',
  port: '8080',
  hostname: 'localhost',
  hash: '#test',
  search: '?year=2025&month=feb',
  query: 'year=2025&month=feb',
  pathname: '/default.html',
  path: '/default.html?year=2025&month=feb',
  href: 'http://localhost:8080/default.html?year=2025&month=feb#test'
}
```

Example-2

Write a Node.js program to **parse a URL** and convert the query string into a **JavaScript object**.

```
var u=require("url");
var addr="http://localhost:8080/default.html?year=2025&month=feb#test";
var q=u.parse(addr,true);
console.log(q);
```

Output:

```
Url {
  protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'localhost:8080',
  port: '8080',
  hostname: 'localhost',
  hash: '#test',
  search: '?year=2025&month=feb',
  query: [Object: null prototype] { year: '2025', month: 'feb' },
  pathname: '/default.html',
  path: '/default.html?year=2025&month=feb',
  href: 'http://localhost:8080/default.html?year=2025&month=feb#test'
}
```

Example-3

Write node js script to fetch the querystring from the url and check if year is leap year or not.

```
var u=require("url");
var addr="http://localhost:8080/default.html?year=2025&month=feb";
var q=u.parse(addr,true);
var qdata=q.query;
console.log(qdata.year);
if(qdata.year%4==0)
{
  console.log("ts a leap year")
}
else{
```

```
console.log("Its not a leap year")
}
```

Example-4

Scenario 1 : Fetch query as an object. Need to convert query to string using stringify and then we can write it in file.

```
var u=require("url");
var ps=require("fs");
var adr1=" http://localhost:8080/default.html?year=2025&month=feb";

var q1=u.parse(adr1,true);
var qdata=JSON.stringify(q1.query);
ps.writeFile("fsd2.txt",qdata,(err)=>
{
  console.log("completed");
});
```

Scenario 2: Fetch query as a string and we can write it in file.

```
var u=require("url");
var ps=require("fs");
var adr1=" http://localhost:8080/default.html?year=2025&month=feb";

var q1=u.parse(adr1,false);
var qdata=q1.query;
ps.writeFile("fsd2.txt",qdata,(err)=>
{
  console.log("completed");
});
```

Example-5

Write a Node.js program to parse a URL and calculate the average of three numbers passed as query parameters.

url: "http://localhost:8080/test?m1=50&m2=60&m3=70";

```
var url = require("url");
process.noDeprecation = true;
```

```

var adr = "http://localhost:8080/test?m1=50&m2=60&m3=70";

var q = url.parse(adr, true);

var m1 = parseInt(q.query.m1);
var m2 = parseInt(q.query.m2);
var m3 = parseInt(q.query.m3);

var avg = (m1 + m2 + m3) / 3;

console.log("Average:", avg);

```

While running above codes of url using `.parse()` it will give below warning.

DeprecationWarning: ``url.parse()`` behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for ``url.parse()`` vulnerabilities.
(Use ``node --trace-deprecation ...`` to show where the warning was created)

Deprecation means a feature is **old and not recommended to use**.

It still works, but may be **removed in future versions**

Option	Command / Code	Purpose
Disable Warning (Terminal)	<code>node --no-deprecation filename.js</code>	Hides deprecation warnings while running program
Suppress in Code	<code>process.noDeprecation = true;</code>	Disables warnings inside the program
Show Full Trace	<code>node --trace-deprecation filename.js</code>	Shows where the warning is generated (for debugging/teaching)

`process.noDeprecation = true;` is used to disable deprecation warnings, so the warning message is not shown during program execution.

HTTP Module

The HTTP module is a built-in module in Node.js that allows creating web servers and handling client requests and responses over HTTP (HyperText Transfer Protocol).

What is HTTP?

HTTP (HyperText Transfer Protocol) is used for communication between:

- Client (browser)
- Server (Node.js)

When a user requests a webpage, the browser sends a request and the server sends a response.

Example:

```
var http = require('http');  
// create a server object  
http.createServer(function (req, res) {  
  // Step 1: Set status + headers  
  res.writeHead(200, { 'Content-Type': 'text/html' });  
  // Step 2: Write response body  
  res.write('<h1>Hello World!</h1>');  
  // Step 3: End response  
  res.end();  
}).listen(3120, () => {  
  console.log("Server is running at http://localhost:3120");  
});
```

Import module

```
var http = require('http');
```

You're using Node's built-in **HTTP module**

This lets you create a web server

Create server

```
http.createServer(function (req, res) {
```

This runs **every time a request comes in**

- req → request from browser

- res → response you send back

writeHead()

```
res.writeHead(200, { "Content-Type": "text/html" });
```

What this does:

1. Status Code

200

Means: **OK / Success**

2. Headers

```
{ "Content-Type": "text/html" }
```

Tells browser:

"I am sending HTML, render it properly"

Without this:

- Browser may treat it as plain text
- HTML tags may not render

Write body

```
res.write("<h1>Hello World!</h1>");
```

This is the **actual data sent to browser**

- You can call .write() multiple times
- It sends data in chunks

End response

```
res.end();
```

This tells Node:

"I'm done. send everything now"

Flow:

Client (Browser) → Request → Server

Server:

writeHead() → set status + headers

write() → send data

end() → finish response

Response goes back to browser

Listening to Port

```
}).listen(3120);
```

The server listens on port 3120.

Run the code through **node file.js** and you can access it in the browser using:

http://localhost:3120



Content-Type header is mandatory for correct rendering. Without it, the browser guesses the format, which can lead to inconsistent results.

Browser does NOT read your mind. It depends on headers to understand data.

- Without header → unlabeled box
→ “What is this? Text? HTML? Image?”
- With header → labeled box
→ “This is HTML → render it”

Add an HTTP Header

Name	MIME type
Plain text	text/plain
HyperText Markup Language (HTML)	text/html
JavaScript Object Notation (JSON)	application/json
JPEG Image	image/jpeg
Portable Network Graphics (PNG)	image/png

Plain text example

```
var http = require('http');

http.createServer( (req, res) => {
  // Tell browser this is plain text
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('<h2>Hello World!</h2>');
}).listen(8180);
```

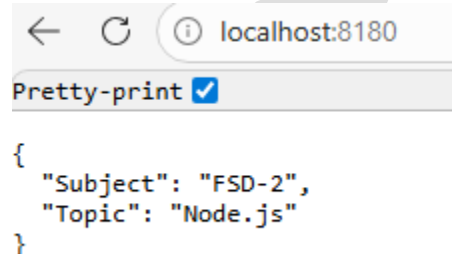
Output:

<h2>Hello World!</h2>

Browser shows **raw text**, not HTML

JSON Example

```
var http = require('http');
http.createServer(function (req, res) {
  // Tell browser this is JSON
  res.writeHead(200, { 'Content-Type': 'application/json' });
  const data = {
    Subject: "FSD-2",
    Topic: "Node.js"
  };
  res.end(JSON.stringify(data));
}).listen(8180);
```



```
{
  "Subject": "FSD-2",
  "Topic": "Node.js"
}
```

Request Url

When a client (browser) sends a request to the server, Node.js provides a req object that contains details about that request. This object is called an http.IncomingMessage. One important property of this object is req.url. It contains the part of the URL that comes after the domain name.

For example, if the user visits:

```
http://localhost:8280/about
```

Then:

```
req.url = "/about"
```

In the given code, the server sends back the value of req.url in the response. This means whatever path the user requests will be displayed in the browser.

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(req.url + "<br>");
  res.end("Url Fetched");
}).listen(8280);
```

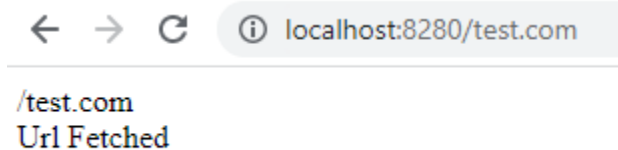


Image example

```
var http = require('http');
http.createServer(function (req, res) {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Hello</h1>');
  }
}).listen(8180);
```

Image is Broken in above example

- The img tag does not contain the image itself. It tells the browser to request the image separately from the server.

- When the page loads, the browser first requests the main page using /. The server sends the HTML response. Then the browser sees the image tag and makes another request for /logo192.png.
- If the server does not handle the /logo192.png request, it does not send any response. Because of this, the browser cannot load the image and shows it as broken.

Solution is below code in above example

```
var http = require('http');
var fs = require('fs');

http.createServer(function (req, res) {

  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Hello</h1>');
  }

  else if (req.url === '/logo192.png') {
    const img = fs.readFileSync('logo192.png');
    res.writeHead(200, { 'Content-Type': 'image/png' });
    res.end(img);
  }

  else{
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end("<h1>Page not found</h1>");
  }

}).listen(8180);
```

- The server must handle the image request and send the image file with the correct Content-Type (image/png).

Each resource like HTML, image, CSS, or JavaScript is a separate HTTP request and must be handled by the server.

Write node js script to perform tasks as asked.

- 1) Create one page with two links (Home(/) and about(/about)).**
- 2) Both pages must contain HTML type content and add required content on both the pages.**
- 3) If user add any other URL path, then he/she will be redirected to page and plain message will be displayed of "Page not found".**

```
var h=require("http");
var server=h.createServer(
  function(req,res)
  {
    if(req.url=="/")
    {
      res.writeHead(200,{"content-type":"text/html"});
      res.write("<h1> Home page </h1><div><ul><li><a href='/'>Home</a></li><li><a href='/about'>About</a></li></ul>");
      res.end();
    }
    else if(req.url=="/about")
    {
      res.writeHead(200,{"content-type":"text/html"});
      res.write("<h1> About Page </h1>");
      res.end();
    }
    else
    {
      res.writeHead(404,{"content-type":"text/plain"});
      res.write("Page not found");
      res.end("\nPlease check the url");
      /* res.write("Bye");*/ //display nothing if you add any content after res.end and throws
      an error in console
    }
  });
server.listen(5051);
```

Write node js script to request server to display JSON data on browser

```
var http=require("http");
var server=http.createServer((req,res)=>{
  if(req.url=="/")
  {
    const a={"Name":"ABC", "Age":35};
    res.writeHead(200,{"content-type":"application/json"});
    res.write("Thank you..!");
    res.write(JSON.stringify(a));
    res.end();
  }
});
server.listen(6001);
```

Output: Thank you..!{"Name":"ABC","Age":35}

Write a nodejs program load a simple html file defined as static on nodejs web server and print its content as html content.

```
var http = require("http");
var fs = require("fs");
var url = require("url");
var addr = "http://localhost:6051/7.html";
var q = url.parse(addr);
http.createServer(function (req, res) {
  fs.readFile("." + q.pathname, function (err, data) {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("File Not Found");
    } else {
      res.writeHead(200, { "Content-Type": "text/html" });
      res.end(data);
    }
  });
}).listen(6051);
```

Write a nodejs program load a simple html file on nodejs web server and print its content as html content.

```
var http = require("http");
var fs = require("fs");
http.createServer(function (req, res) {
  var file = "." + req.url;
  fs.readFile(file, function (err, data) {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("File Not Found");
    } else {
      res.writeHead(200, { "Content-Type": "text/html" });
      res.write(data);
      res.write("<h1>Hello</h1>");
      res.end();
    }
  });
}).listen(6055);
```

Explanation

- Uses req.url to load file dynamically
- / loads default file (7.html)
- Other URLs load corresponding files
- Adds extra HTML (Hello) in response

Write node js script to fetch values from url given below and display output as asked.

"https://www.google.com/exam.txt?c1=Hello&c2=FSD2 T1 Test&c3=Welcome to LJU#AllTheBest"

1) Data must be written as below in file named "exam.txt". File name must be fetched from the url given above.

Output:

Hello!

Welcome to LJU FSD2 T1 Test

#AllTheBest

2) Read content from file "exam.txt" and send response to server and display data in "/" page in same format as above but in H1 tag and in red color.

3) If any other page is requested it shows "Page not found" message in plain text.

```
const http = require("http");
const fs = require("fs");
const url = require("url");

const url1 =
"https://www.google.com/exam1.txt?c1=Hello&c2=FSD2+T1+Test&c3=Welcome+to+LJU#AllTheBest";

const parsedUrl = url.parse(url1,true);
console.log(parsedUrl.pathname)

const c1 = parsedUrl.query.c1;
const c2 = parsedUrl.query.c2;
const c3 = parsedUrl.query.c3;
const hash = parsedUrl.hash;
const filename = "."+parsedUrl.pathname

const fileContent = c1+"!\n"+c3+" "+c2+"\n"+hash;

fs.writeFileSync(filename, fileContent);

const server = http.createServer((req, res) => {
  if (req.url === "/" ) {
```

```

    data= fs.readFileSync(filename, "utf-8")
    res.writeHead(500, { "Content-Type": "text/html" });
    res.end("<h1 style='color:red'><pre>"+data+"<pre></h1>");
  }
  else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page not found");
  }
});

server.listen(3000)

```

Create HTTP webpage on which Home page display “Welcome to Log in page” in blue color and font size must be 32px, Login page shows one HTML file from static URL having Form with detail for Username, Password, submit and reset button, Gallery page reflect one Image “hello.png” and any other page shows “Page Not found”. Write all necessary files to perform task. (Image already exist in same folder)

```

var http = require("http");
var fs = require("fs");

var server = http.createServer(function (req, res) {

  // Home Page
  if (req.url == "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.write("<h1 style='color:blue;font-size:32px;'>Welcome to Log in page</h1>");
    res.end();
  }

  // Login Page
  else if (req.url == "/login") {
    var d= fs.readFileSync("login.html");
    res.writeHead(200, { "Content-Type": "text/html" });
    res.write(d);
    res.end();
  }
}

```

```
// Gallery Page
else if (req.url == "/gallery") {
  var d= fs.readFileSync("hello.png")
  res.writeHead(200, { "Content-Type": "image/png" });
  res.end(d);
}

// Page Not Found
else {
  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Page Not Found");
}

});

server.listen(5051, () => {
  console.log("Server running at http://localhost:5051");
});
```